JavaScript-04 第12回_まとめ

第12回の目標

- **Swiperの導入の流れを理解する**
- **Swiperでスライドショーを作る設定を理解する（自動再生・fade）**
- **Swiperでカルーセルを作る仕組みを理解する（レスポンシブ含む）**
- **アコーディオンメニューの仕組みを理解する（forEachの復習）**

Swiper

Swiperとは

- スライドショーやカルーセル（横にスクロールするスライドギャラリー）を作れるJavaScriptのプラグイン
- オプションを指定するだけで、自動再生・ループ・前後ボタン・レスポンシブなどを設定できる
- Swiper公式／Swiper公式のデモページ

導入（CDN）

Swiper公式の「Get Started」→「Use Swiper from CDN」から読み込みリンクをコピーして、headタグ内に貼る。

```
<!-- SwiperのCSS -->
<link rel="stylesheet" href="https://cdn.jsdelivr.net/npm/swiper@8/swiper-bundle.min.css" />

<!-- SwiperのJS → 自分のJSの順で読み込む -->
<script src="https://cdn.jsdelivr.net/npm/swiper@8/swiper-bundle.min.js"></script>
<script src="js/main.js" defer></script>
```

- 必要なファイルはCSS（`swiper-bundle.min.css`）とJS（`swiper-bundle.min.js`）の2つ
- `@8` の部分がバージョン。変えたいときは`@`の後の数字を変えるだけでよい
  - バージョン7・8あたりが安定していると言われ、実務でもよく使われる（教材はバージョン8）
- Swiperを使うHTMLファイルが複数あるときは、それぞれのファイルに同じように貼る
- ローカルにダウンロードして読み込む方法もある（`css/swiper-bundle.min.css`、`js/swiper-bundle.min.js` のように配置する）

HTMLの基本構造

```
<div class="swiper">                    <!-- スライダー全体の入れ物（必須） -->
  <div class="swiper-wrapper">          <!-- スライドを囲む要素（必須） -->
    <div class="swiper-slide">1枚目</div> <!-- スライド1枚ぶん（必須） -->
    <div class="swiper-slide">2枚目</div>
    <div class="swiper-slide">3枚目</div>
  </div>
  <div class="swiper-pagination"></div>  <!-- ページネーション（省略可） -->
  <div class="swiper-button-prev"></div> <!-- 前へボタン（省略可） -->
  <div class="swiper-button-next"></div> <!-- 次へボタン（省略可） -->
  <div class="swiper-scrollbar"></div>   <!-- スクロールバー（省略可） -->
</div>
```

クラス	役割
`swiper`	スライダーのメインコンテナ
`swiper-wrapper`	スライドを囲む要素
`swiper-slide`	個々のスライド
`swiper-pagination`	ページネーション（現在位置を示すドットなど）
`swiper-button-prev` / `swiper-button-next`	前へ／次へボタン
`swiper-scrollbar`	スクロールバー

基本の書き方

```
const 定数名 = new Swiper(要素, { カスタマイズ });
```

- 第1引数に、スライダーにする要素（またはクラス名の文字列）を渡す
- 第2引数の`{ }`の中に「オプション名: 値」を書く。複数あるときはカンマで区切る
- 定数名は自由だが、何を表しているか分かる名前にする

教材では、要素が存在するときだけSwiperを動かす書き方をしている。

```
// .js_sliderクラスを持つ要素を取得して、定数sliderDomに代入する
const sliderDom = document.querySelector(".js_slider");

// もしsliderDomが存在したら、Swiperを作る
if (sliderDom) {
  const slider = new Swiper(sliderDom, {
```

```
    loop: true, // 最後までいったら最初に戻る
  });
}
```

主なオプション

オプション	値の例	意味
`loop`	`true` / `false`	スライドを無限ループさせる
`autoplay`	`true` / `{ delay: 1600 }`	自動再生する
`speed`	`1200`	切り替えアニメーションの速さ（ミリ秒、初期値は300）
`effect`	`"slide"` / `"fade"` / `"cube"` / `"coverflow"` / `"flip"`	切り替え時の動きの種類
`initialSlide`	`0`, `1` …	最初に表示するスライドの番号（0から数える）
`slidesPerView`	`3`	同時に表示する枚数
`spaceBetween`	`20`	スライド間の余白（px）
`pagination`	`{ el: ".swiper-pagination" }`	ページネーションの設定
`navigation`	`{ nextEl: …, prevEl: … }`	前へ／次へボタンの設定
`scrollbar`	`{ el: ".swiper-scrollbar" }`	スクロールバーの設定
`breakpoints`	`{ 768: { … } }`	画面幅ごとに設定を切り替える（レスポンシブ）
`allowTouchMove`	`false`	スワイプ・タッチ操作で動かせるかどうか

`autoplay`・`pagination`・`navigation`の中で使うオプション

オプション	意味
`autoplay` → `delay`	次のスライドに切り替わるまでの時間（ミリ秒、1000 = 1秒）
`autoplay` → `disableOnInteraction`	ユーザーが操作したら自動再生を止めるか（`false`で止めない）
`pagination` → `el`	ページネーションを表示する要素
`pagination` → `clickable`	ページネーションのドットをクリックできるか
`pagination` → `type`	表示形式（`bullets` / `fraction` / `progressbar` / `custom`）

オプション	意味
`navigation` → `nextEl` / `prevEl`	次へ／前へボタンにする要素

- オプションは全部覚えなくてよい。必要なときに公式ドキュメントや「swiper オプション」で調べる
- 【Swiper】カスタマイズ用オプションまとめ一覧

スライドショー（effect: fade / autoplay）

フェードで切り替わりながら自動再生する。

```
const slideshowDom = document.querySelector(".js_slideshow");

if (slideshowDom) {
  const slideshow = new Swiper(slideshowDom, {
    speed: 1200,      // 切り替えアニメーションの速さを1.2秒にする
    effect: "fade",   // 切り替え効果をフェードにする
    loop: true,       // 最後までいったら最初に戻る
    autoplay: {
      delay: 1600,                // 1.6秒ごとに次のスライドへ切り替える
      disableOnInteraction: false, // ユーザーが操作しても自動再生を止めない
    },
  });
}
```

- 自動再生するだけなら`autoplay: true`でもよい。時間などを細かく決めたいときは`{ }`で中身を指定する

カルーセル（slidesPerView / spaceBetween / navigation / pagination）

複数枚を並べて表示し、前後ボタンで送る。

```
const carouselDom = document.querySelector(".js_carousel");

if (carouselDom) {
  const carousel = new Swiper(carouselDom, {
    loop: true,          // 最後までいったら最初に戻る
    slidesPerView: 3,    // 3枚同時に表示する
    spaceBetween: 20,    // スライド間の余白を20pxにする
    pagination: {
      el: ".swiper-pagination", // ページネーションを表示する要素
```

```
      clickable: true,          // ドットをクリックできるようにする
    },
    navigation: {
      nextEl: ".swiper-button-next", // 次へボタンの要素
      prevEl: ".swiper-button-prev", // 前へボタンの要素
    },
  });
}
```

レスポンシブ（breakpoints）

- `breakpoints`のキーに画面幅（px）を書くと、**その幅以上**のときに中の設定が適用される
- `breakpoints`の外に書いた設定が、一番小さい画面幅での設定になる

```
const carousel = new Swiper(".js_carousel", {
  slidesPerView: 1,   // 0〜767px：1枚表示
  spaceBetween: 20,
  breakpoints: {
    768: {
      slidesPerView: 2,      // 768px以上：2枚表示
    },
    1088: {
      slidesPerView: 3,      // 1088px以上：3枚表示
      allowTouchMove: false, // スワイプ・タッチ操作で動かせないようにする
    },
  },
});
```

Swiperが動かないときの対処法

原因	確認すること
ファイルを読み込んでいない	CSS（`swiper-bundle.min.css`）とJS（`swiper-bundle.min.js`）の**両方**を読み込んでいるか
読み込みが正しくない	CDNのURLやローカルファイルのパスが間違っていないか
読み込みの順番が違う	Swiper本体 → 自分のJS（main.js）の順か。headに書くときはmain.jsに`defer`をつけているか

原因	確認すること
HTMLの必須要素がない	`swiper`・`swiper-wrapper`・`swiper-slide`の3つがあるか
JSの記述ミス	`new Swiper`に渡すクラス名、オプション名のつづり、カンマ忘れ、値の書き方（`true`/`false`、数値、文字列の引用符）

- Swiperが動かないときの対処法

アコーディオンメニュー

アコーディオンメニューとは

- ナビゲーションメニューの1つ
- 項目をクリック・タップすると、隠れている詳細が表示される
- もう一度クリックすると、元のように隠れる
- よく使われるのはFAQ（質問をクリックすると解答が開く）

仕組み（クラスで開閉状態を管理する）

「今開いているか」をクラスの有無で判断し、見た目の変化もクラスで切り替える。

要素	役割	つけるクラス
`.js_faq`	1つの質問と解答のまとまり（クリック対象）	開いているとき`is-active`
`.js_faq-a`	開閉させる中身（解答）	—
`.js_faq_mark`	「＋」「－」のアイコン	開いているとき`is-open`

処理の流れ 1. `.js_faq`を`querySelectorAll`ですべて取得する 2. `forEach`で1つずつ取り出し、それぞれの中から`.js_faq-a`を取得する 3. クリックイベントをつける 4. `is-active`がついているかを`if`文で判定する - ついている（開いている）→ クラスを外して、閉じるアニメーションを実行 - ついていない（閉じている）→ クラスをつけて、開くアニメーションを実行 5. アニメーションはGSAPで、開く処理と閉じる処理を別々の関数に分けておく

forEachで複数の要素に処理をつける

第11回の復習。配列（`querySelectorAll`で取得した要素の集まり）に対して、要素の数だけ処理を繰り返す。

```
配列.forEach(function (要素, 番号) {
  // 要素1つずつに対して行う処理
});
```

- 第1引数：ループのたびに、配列の要素が順番に入る
- 第2引数：要素の番号（0から始まる）。省略できる

```
// .js_faqクラスを持つ要素をすべて取得する
const faq = document.querySelectorAll(".js_faq");

// faqの要素を1つずつ取り出し、elementとして処理する
faq.forEach(function (element) {
  // element（今処理している.js_faq）の中から.js_faq-aを取得する
  const faqA = element.querySelector(".js_faq-a");
});
```

- `document.querySelector` ではなく `element.querySelector` にすることで、「どのアコーディオンの中身か」を指定できる
- JavaScriptのループ処理を学ぼう

if文とclassListで開閉を判定する

第9回の復習。

メソッド	意味
`classList.contains("クラス名")`	クラスがついているか調べる（`true` / `false` を返す）
`classList.add("クラス名")`	クラスを追加する
`classList.remove("クラス名")`	クラスを取り除く
`classList.toggle("クラス名")`	クラスがあれば外し、なければつける

```
// elementがクリックされたら、中の処理を実行する
element.addEventListener("click", function () {
  // もしelementにis-activeがついていたら（開いていたら）
  if (element.classList.contains("is-active")) {
    element.classList.remove("is-active");                                  // is-activeを外す
    element.querySelector(".js_faq_mark").classList.remove("is-open");   // アイコンを「＋」に戻す
    closingAnim(faqA);                                                      // 閉じるアニメーションを実行
  } else {
```

```
    element.classList.add("is-active");                                    // is-activeをつける
    element.querySelector(".js_faq_mark").classList.add("is-open");        // アイコンを「-」にする
    openingAnim(faqA);                                                     // 開くアニメーションを実行
  }
});
```

- `if`の中では`add`/`remove`の代わりに`toggle`を使っても同じ動きになる
- JavaScriptでクラスを操作しよう／JavaScriptの条件分岐を学ぼう

GSAPで開閉アニメーションをつける

第10回の復習。開く処理と閉じる処理を関数に分け、動かす要素（中身）を引数で受け取る。

```
// 閉じるアニメーション：contentを高さ0・透明にする
const closingAnim = function (content) {
  gsap.to(content, {
    height: 0,             // 高さを0にする
    autoAlpha: 0,          // 透明にする
    duration: 0.4,         // 0.4秒かけて変化させる
    ease: "Power4.inOut", // 滑らかに加速して減速する
  });
};

// 開くアニメーション：contentを中身の高さ・不透明にする
const openingAnim = function (content) {
  gsap.to(content, {
    height: "auto",        // 中身の高さに合わせて広げる
    autoAlpha: 1,          // 不透明にする
    duration: 0.4,
    ease: "Power4.inOut",
  });
};
```

- 終わりの状態だけ指定すればよいので`gsap.to()`を使う

メソッド	指定するもの	考え方
`gsap.to()`	今 → 未来	今の状態からどうなりたいか
`gsap.from()`	過去 → 今	どの状態から今の状態になりたいか

メソッド	指定するもの	考え方
`gsap.fromTo()`	過去 → 未来	どの状態からどの状態になりたいか

【別の書き方】GSAPを使わずにクラスとCSSで開閉する

- 参考記事では、`classList.toggle("is-open")`でクラスを切り替え、見た目の変化はCSSの`transition`でつけている
- クリックしたボタンの次の要素（中身）は`this.nextElementSibling`で取得している
- jQueryを使わないアコーディオンメニューの作り方

JSがうまく動かないとき

エラーの確認手順

1. コンソール画面でエラーを確認する（エラーは直すためのヒント）
2. 原因を調べる
    - エラー文で検索する（例：「◯◯（エラー文） 原因」）
    - AIにエラー文の解説と対策を聞く
3. エラーが出ていないとき
    - スーパーリロードする（mac：`command + Shift + R`／win：`control + Shift + R`）。JSは普通のリロードでは反映されないことがある
    - `console.log("ok!")`を処理の区切りに書き、どこまで動いているか確かめる

- AIにはコードを書かせず、理解の補助に使う。書かせ続けると、困ったときに自分で直せなくなる

よくあるエラーの種類

エラー	意味
`SyntaxError`	文法ミスがある
`TypeError`	型が正しくない
`ReferenceError`	存在しない（定義していない）変数を参照した
`RangeError`	範囲外の値を使った

- JavaScriptでよく見るエラーとその対策

頻出UIパーツ

以下をVanillaJS（素のJavaScript）かGSAPで作れるようになるのが目標。暗記は不要で、仕組みを理解したうえでコピペしてアレンジできればよい。 - ハンバーガーメニュー（開閉、リンクをクリックしたらメニューを閉じてページ内遷移） - タブメニュー - モーダルウィンドウ - スクロールアニメーション（スクロールしたらふわっと出てくる） - スムーススクロール（「TOPへ戻る」ボタン） - アコーディオンメニュー - スライドショー・カルーセル（Swiper）

「アニメーション名 VanillaJS」で検索して、真似しながら練習する。

参考記事

- 【JavaScript】Swiperの使い方とオプション・カスタマイズ方法について解説
- Swiperの使い方とオプション（ループ・カルーセル・レスポンシブ・動かないときの対処法）
- 【実装例あり】スライダーSwiper.jsの基本的な使い方、機能解説
- Swiperの使い方・カスタマイズを解説！サンプルやオプション15個付き
- 【Swiper】カスタマイズ用オプションまとめ一覧
- Swiperスライダー実装まとめ23選
- Swiperが採用されたサイト事例一覧（Check WebTech）
- 【JavaScript】jQueryを使わないアコーディオンメニューの作り方を解説
- JavaScriptのループ処理を学ぼう
- JavaScriptでクラスを操作しよう
- JavaScriptの条件分岐を学ぼう
- GSAPを使ってアニメーションをつけよう
- JavaScriptでよく見るエラーとその対策